Python 개요와 데이터 분석 환경 구축

한성대학교 컴퓨터공학부
이청용 교수

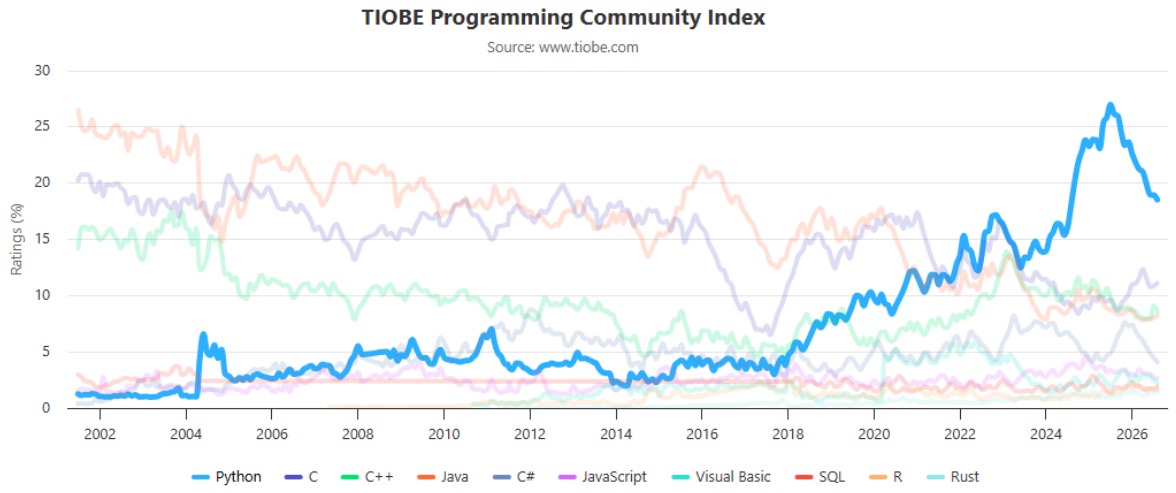
Python 개요

Python의 개요와 데이터 분석 활용

- **Python은 간결하고 읽기 쉬운 문법을 제공하는 오픈소스 범용 프로그래밍 언어**
  - 1991년 Guido van Rossum이 발표한 이후 교육, 연구 및 산업 전반으로 활용 확대
- **Python은 데이터 분석의 전체 과정을 하나의 언어로 처리 가능**
  - 데이터 수집과 정리, 분석, 시각화, 모델 개발까지 지원
  - 대표 라이브러리: NumPy, Pandas, Matplotlib, Scikit-learn, PyTorch 등
- **대학과 산업 현장에서 데이터 분석 및 AI 교육을 위한 주요 언어로 활용**
  - 비교적 쉬운 문법과 풍부한 학습 자료를 바탕으로 입문 교육과 실무 활용에 적합

Python의 범용성과 활용 현황

- **Python은 특정 분야에 제한되지 않고 다양한 SW 개발에 활용 가능**
  - 데이터 분석, AI, 웹 개발, 업무 자동화와 과학 계산 등에 사용
- **데이터 과학과 AI의 성장에 따라 Python 활용 범위 지속 확대**
  - 분석 결과를 응용 프로그램이나 웹 서비스로 연결하기 용이
  - 기존 시스템과 연계하거나 반복 업무를 자동화하는 도구로 활용 가능
- **2026년 8월 기준 TIOBE 프로그래밍 언어 인기 지표에서 Python 1위 기록**
  - Python은 18.53%로 주요 프로그래밍 언어 가운데 높은 관심도 유지



TIOBE Programming Community Index(2026년 8월 기준)

Python의 장점과 라이브러리 생태계

- **간결한 문법을 통해 프로그램의 동작 원리와 데이터 분석 과정에 집중 가능**
  - 코드의 구조를 비교적 쉽게 읽고 작성할 수 있어 프로그래밍 입문에 적합
  - C, Java, R 등 다른 언어의 경험을 Python 학습에 활용 가능
- **목적에 맞는 라이브러리를 조합하여 다양한 문제 해결 가능**
  - 수치 계산: NumPy
  - 데이터 처리 및 분석: Pandas
  - 데이터 시각화: Matplotlib, Seaborn, Plotly
  - 머신러닝 및 딥러닝: Scikit-learn, TensorFlow, PyTorch
- **Python Package Index(PyPI)를 중심으로 방대한 오픈소스 생태계 형성**
  - 2026년 9월 기준 88만 개 이상의 프로젝트 등록 (https://pypi.org/)
  - 필요한 기능을 직접 구현하지 않고 다양한 공개 패키지로 확장 가능

Python의 확장성과 AI 활용

- **데이터 규모에 따라 적절한 처리 도구를 선택하여 분석 환경 확장 가능**
  - NumPy와 Pandas를 활용한 개인용 컴퓨터 중심의 데이터 분석
  - Dask를 활용한 대용량 데이터의 병렬 처리
  - PySpark를 활용한 여러 컴퓨터 기반의 분산 데이터 처리
- **Python은 머신러닝과 생성형 AI 개발에 필요한 주요 생태계 제공**
  - Scikit-learn을 활용한 기본 머신러닝 모델 학습
  - TensorFlow와 PyTorch를 활용한 딥러닝 모델 개발
  - Hugging Face Transformers를 활용한 언어·이미지·음성 모델 활용
- **Python 3.14(2026년 9월 기준)에서는 멀티코어 활용을 위한 실행 방식 개선 지속**
  - 전역 인터프리터 잠금 없이 실행하는 free-threaded Python 공식 지원
  - 일반적인 데이터 분석 실습에서는 기존 실행 방식을 그대로 사용 가능

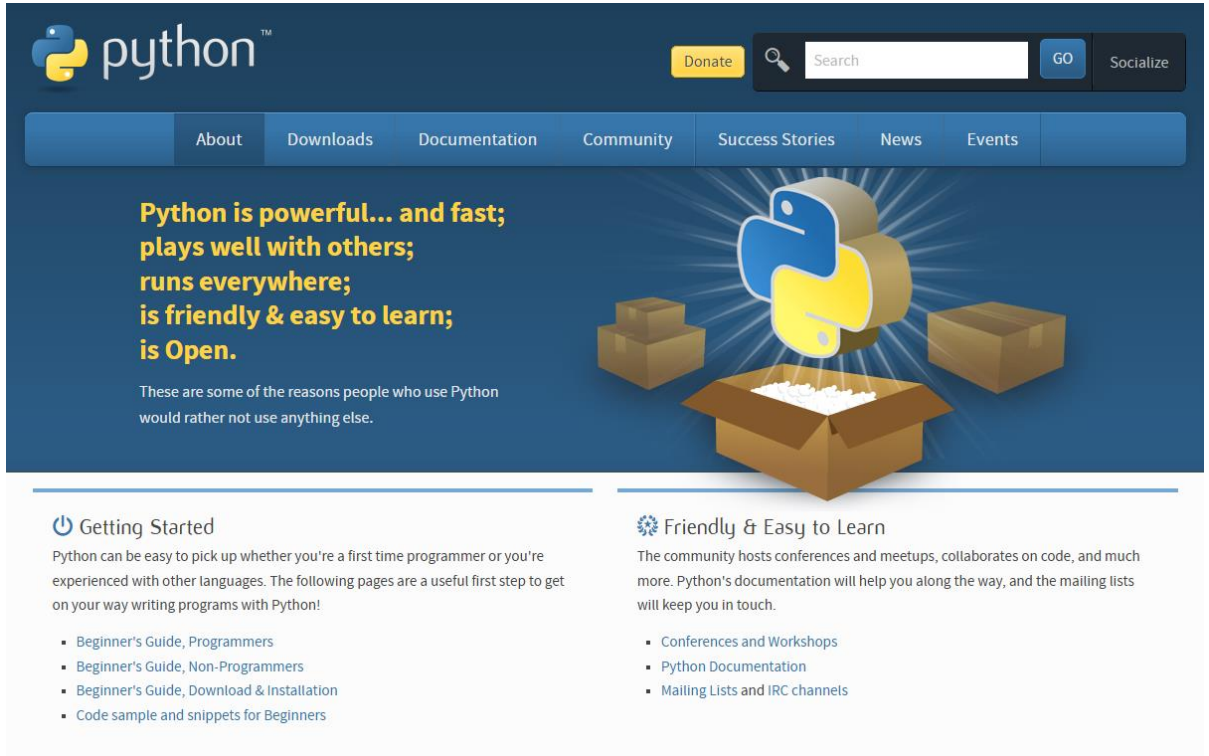
Python의 단점과 한계

- **순수 Python 코드는 C와 C++ 같은 컴파일 언어보다 실행 속도가 느릴 수 있음**
  - 대규모 반복 계산은 NumPy와 같은 최적화 라이브러리로 성능 보완 가능
  - 대용량 데이터는 Dask와 PySpark 등의 병렬·분산 처리 도구 활용 필요
- **라이브러리가 많아 패키지 선택과 버전 관리가 복잡해질 수 있음**
  - 서로 다른 라이브러리가 요구하는 버전 차이로 충돌 발생 가능
  - 프로젝트별 가상환경을 구성하여 Python과 라이브러리 버전 관리 필요
- **모바일 앱, 임베디드 시스템과 엄격한 실시간 처리 분야에서는 활용 제한**
  - 개발 목적과 실행 환경에 따라 다른 언어 또는 도구와의 결합 필요

Anaconda 설치 방법

Python 분석 환경과 Anaconda

- **Python 공식 설치판은 Python 실행기와 표준 라이브러리를 중심으로 구성**
  - 공식 홈페이지(https://www.python.org/)에서 운영체제별 설치 파일 제공
  - 데이터 분석에 필요한 NumPy, pandas, Jupyter 등의 추가 설치 필요
  - 가볍고 일반적인 Python 프로그래밍과 응용 프로그램 개발에 적합

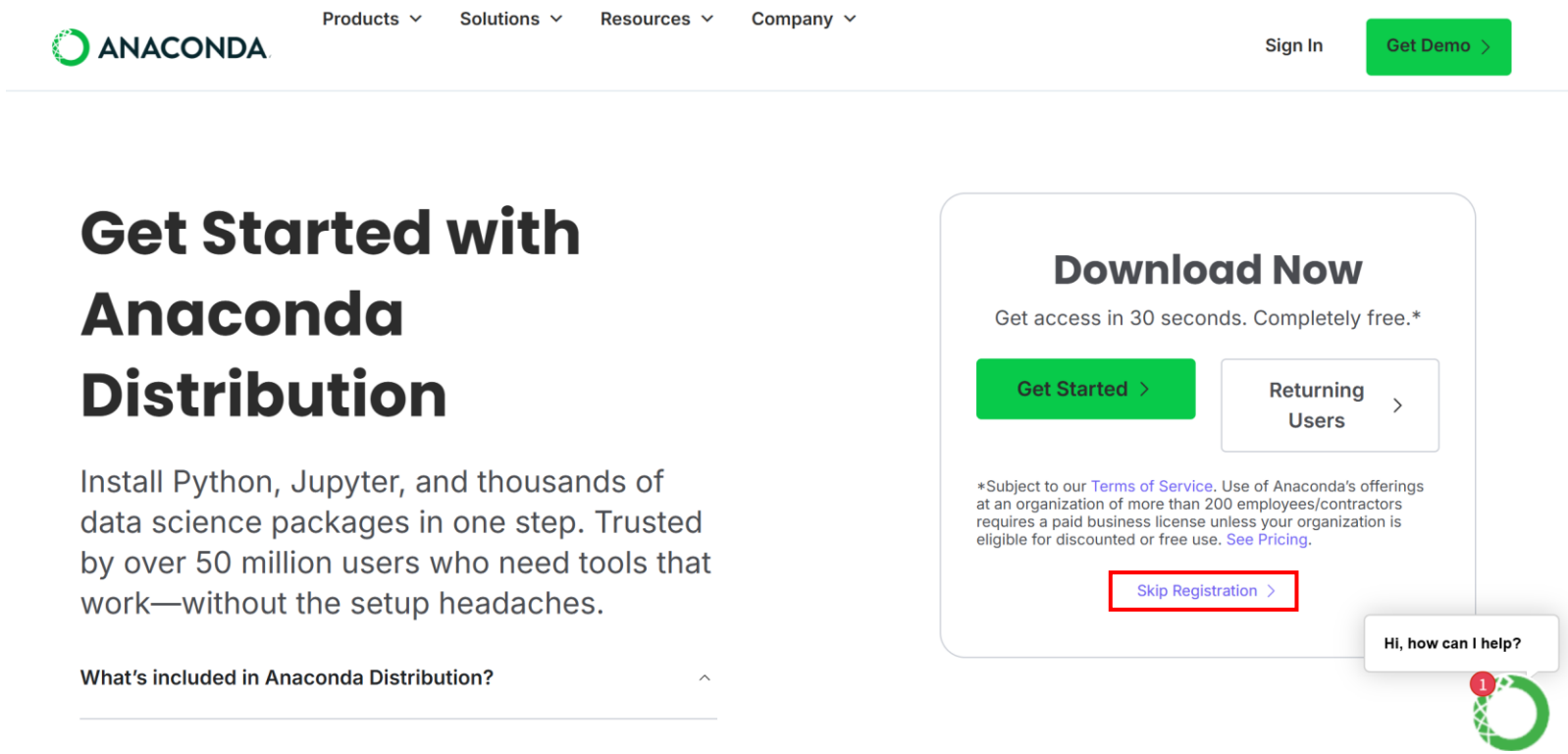


Python 분석 환경과 Anaconda

- **Anaconda Distribution은 데이터 분석에 필요한 주요 도구를 하나의 배포판으로 제공**
  - Python, conda, Anaconda Navigator, Jupyter Notebook과 JupyterLab 포함
  - conda를 이용한 패키지 설치와 프로젝트별 가상환경 관리 가능
  - 추가 패키지는 Anaconda 저장소 등의 채널을 통해 선택적으로 설치 가능
- **본 수업은 학생별 실습 환경의 차이를 줄이기 위해 Anaconda Distribution 사용**
  - 동일한 Python 버전과 라이브러리 구성을 위한 가상환경 생성
  - VS Code와 Jupyter Notebook을 연결하여 실습 진행
  - **본 수업은 Windows 운영체제를 기준으로 설치 방법 설명**

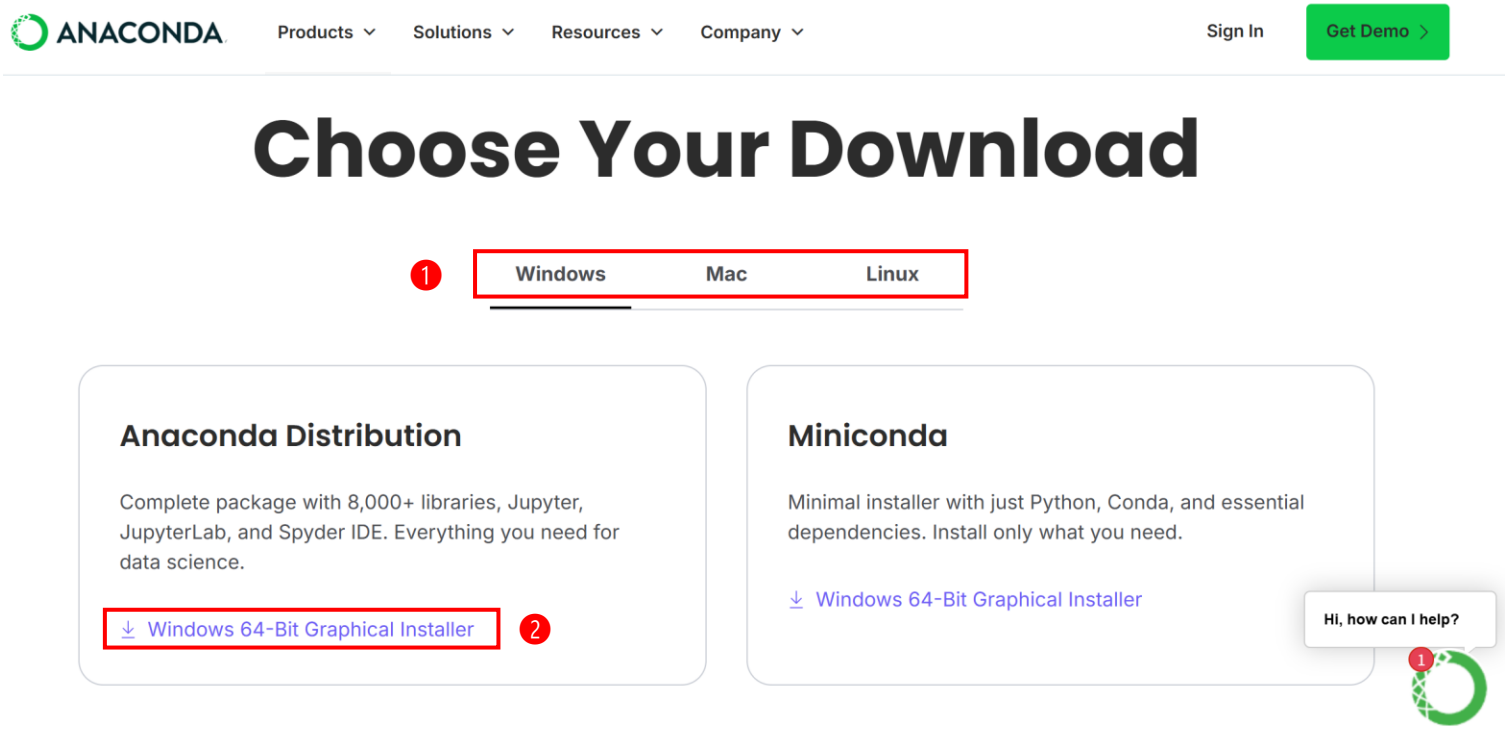
설치 파일 다운로드

- Anaconda 공식 다운로드 페이지 접속
  - https://www.anaconda.com/download
  - 계정 등록을 원하지 않으면 [Skip Registration] 선택



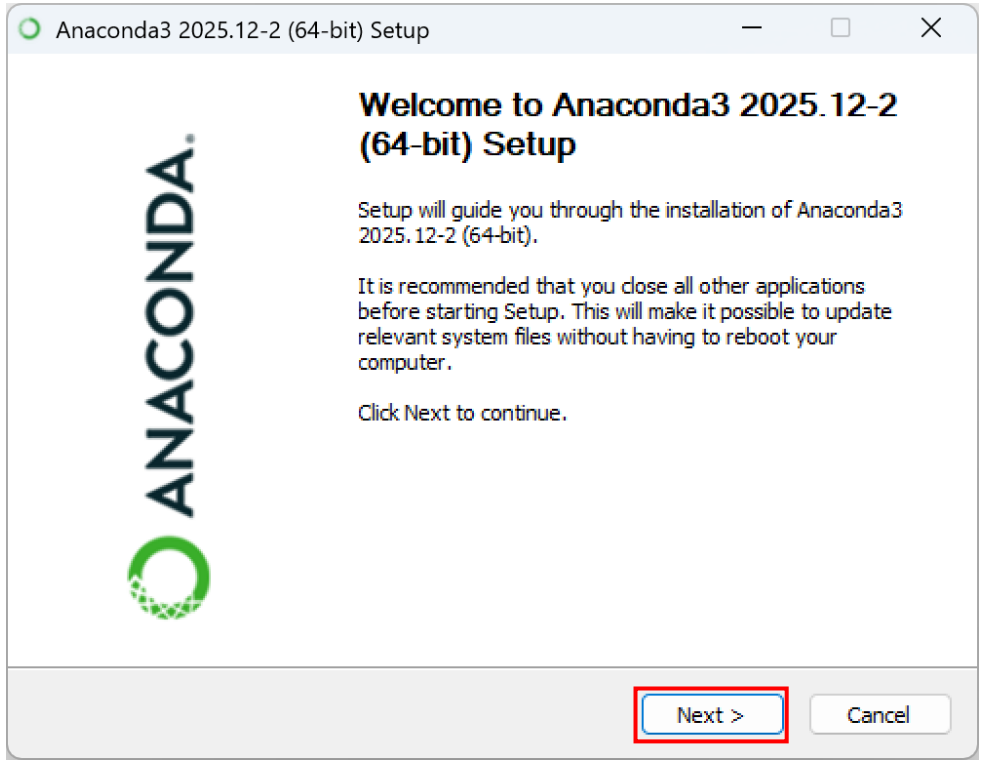
설치 파일 선택

- **설치할 컴퓨터의 운영체제에 맞는 설치 파일 선택**
  - 수업에서는 전체 분석 환경을 제공하는 Anaconda Distribution 사용
  - Miniconda는 conda와 Python 등을 중심으로 제공하는 최소 설치판



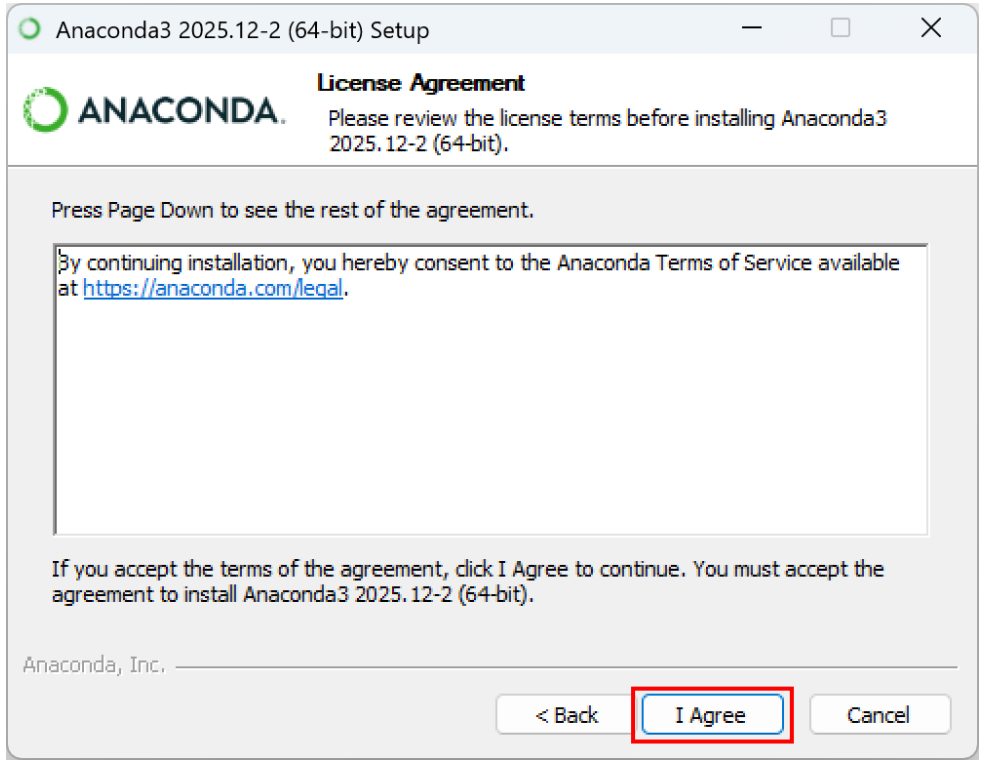
설치 프로그램 실행

- 다운로드한 Anaconda 설치 파일 실행
  - 설치 시작 화면에서 [Next] 선택



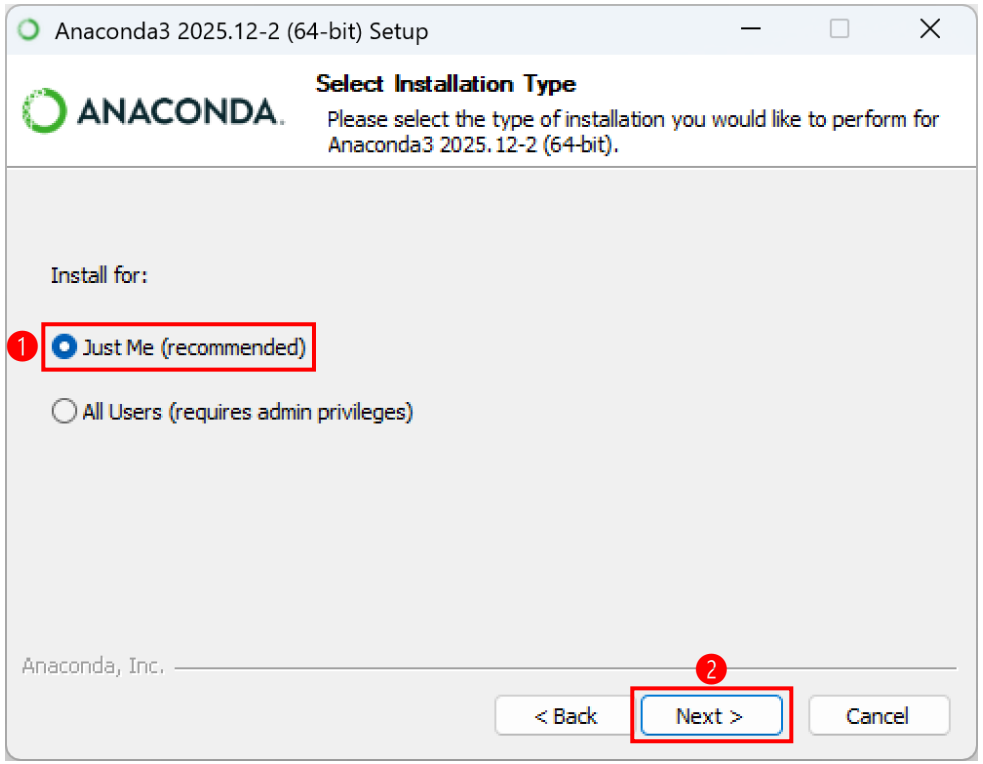
이용약관 동의

- **이용약관 확인 후 [I Agree] 선택**
  - 약관에 동의해야 설치 진행 가능



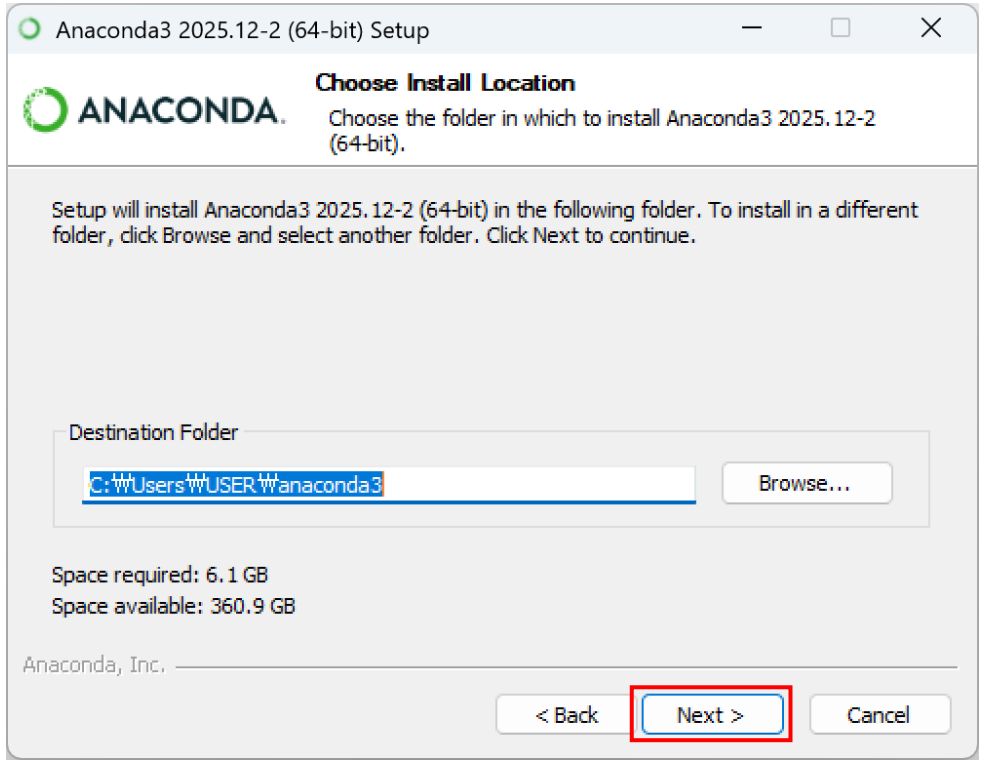
설치 유형 선택

- [Just Me (Recommended)] 선택 후 [Next] 진행
  - 현재 Windows 사용자 계정에만 Anaconda 설치
  - All Users는 시스템 관리가 필요한 경우에만 선택



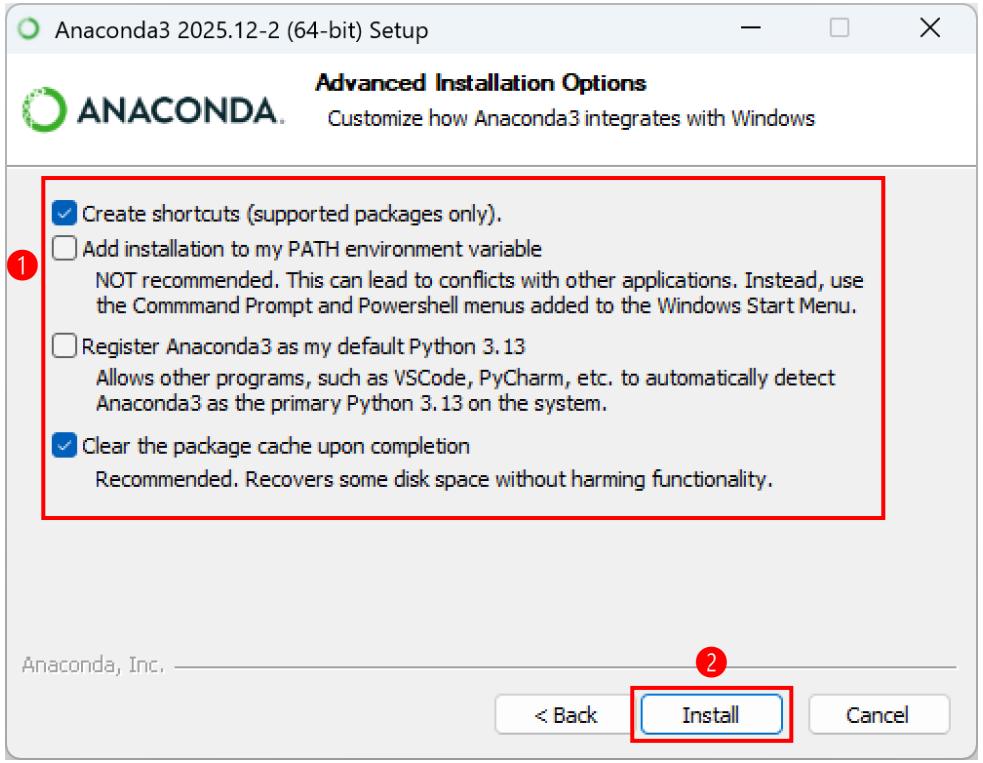
설치 경로 설정

- **설치 경로를 확인한 후 [Next] 선택**
  - 기본 설치 경로에 공백, 한글 및 특수문자가 없으면 그대로 사용
  - Windows 사용자 이름에 한글이나 공백이 포함된 경우 별도 경로 사용 권장
  - 권장 예시: C:\anaconda3



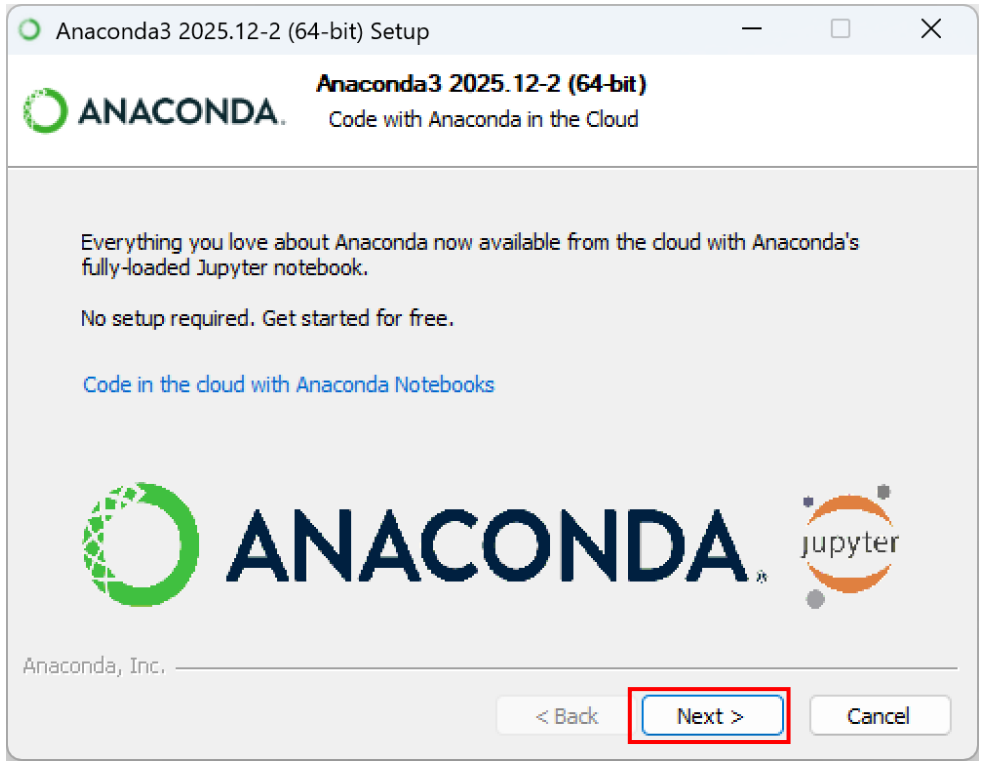
추가 옵션 설정

- **권장 설치 옵션을 확인한 후 [Install] 선택**
  - 시작 메뉴에서 Anaconda 도구를 실행할 수 있도록 [Create shortcuts] 선택
  - 다른 Python 설치판과의 충돌을 방지하도록 [Add Anaconda3 to my PATH] 선택 해제
  - VS Code에서 Anaconda를 기본 Python으로 사용하려면 [Register Anaconda3 as my default Python] 선택
  - 설치 후 저장 공간을 확보하려면 [Clear the package cache upon completion] 선택 권장



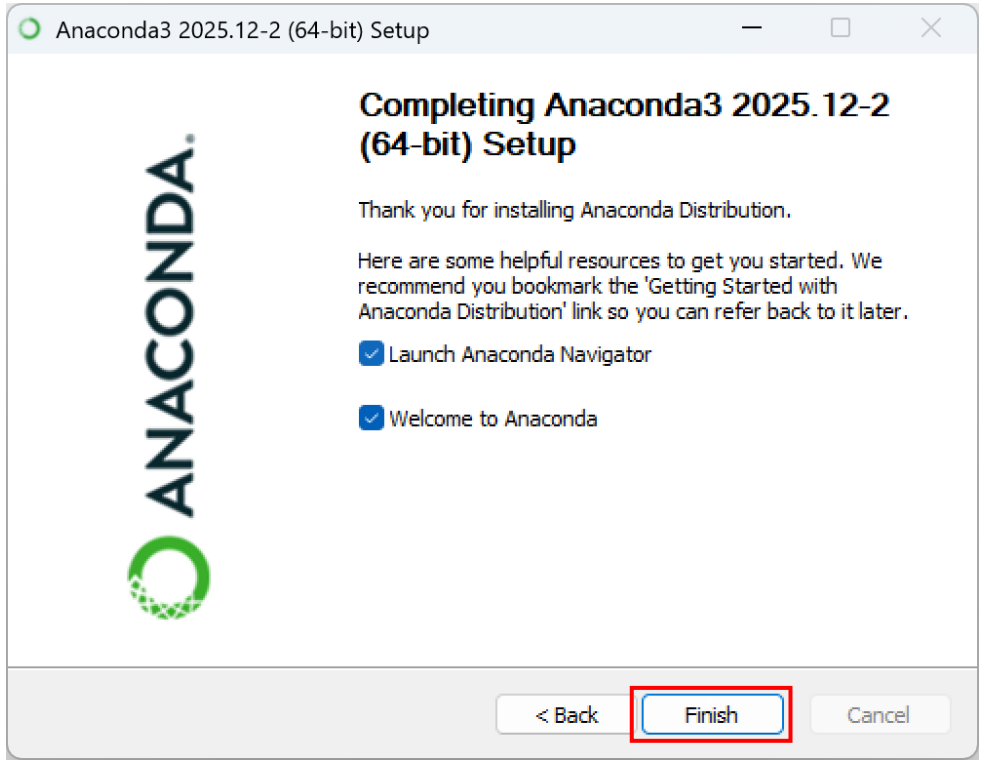
설치 진행

- **[Install] 선택 후 설치 완료까지 대기**
  - 설치가 끝나면 [Next] 선택
  - 추가 안내 화면이 나타나면 다시 [Next] 선택



설치 완료

- **완료 화면에서 실행 옵션을 확인한 후 [Finish] 선택**
  - Windows 시작 메뉴에서 Anaconda Prompt 실행
  - conda --version 명령어를 입력하여 conda 설치와 버전 확인
  - 명령어가 정상적으로 실행되면 Anaconda 설치 완료



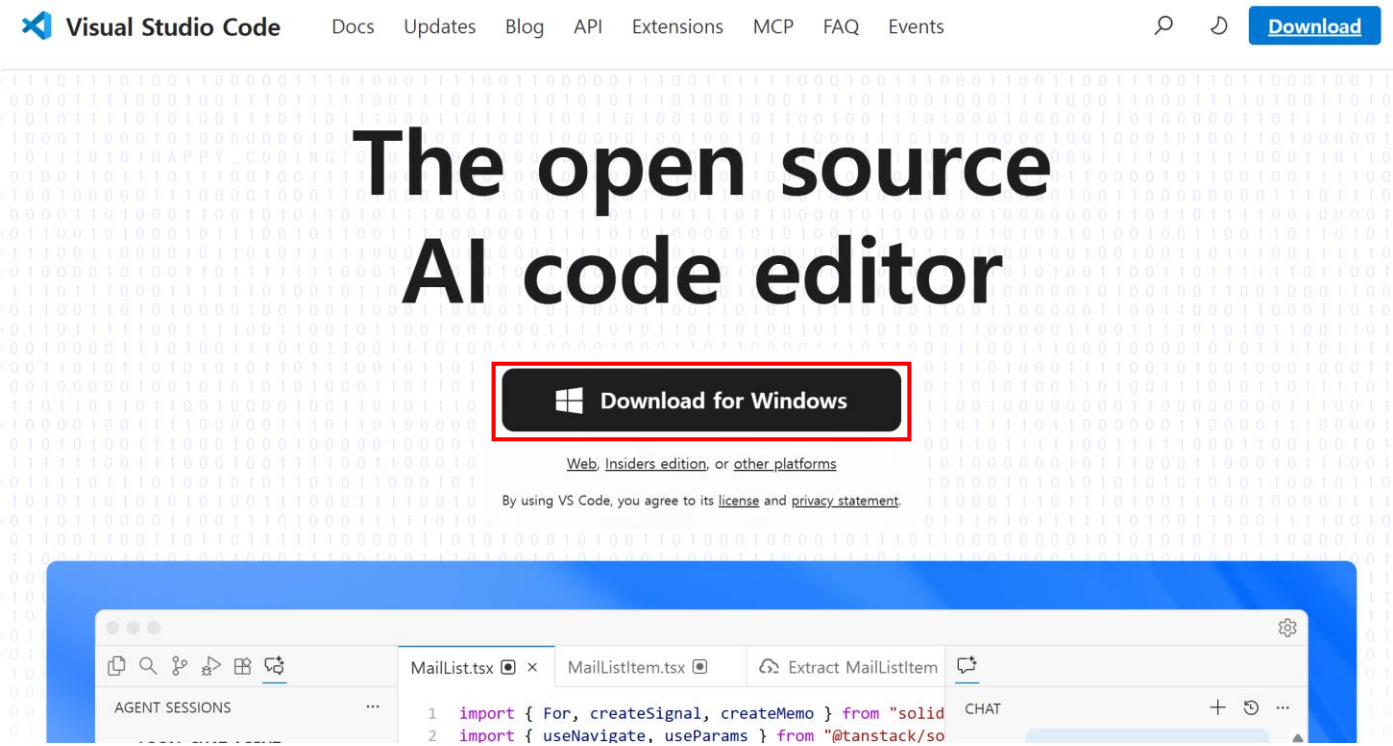
Visual Studio Code 설치 방법

Visual Studio Code 개요

- **Visual Studio Code(VS Code)는 Microsoft에서 제공하는 무료 코드 편집기**
  - 오픈소스 Code – OSS를 기반으로 개발된 Microsoft 제품
  - Windows, macOS, Linux 운영체제 지원
  - 코드 편집, 통합 터미널, 디버깅과 버전 관리 기능 제공
- **확장 기능을 설치하여 프로그래밍 언어와 작업 목적에 맞게 기능 확장 가능**
  - Python 확장을 통한 코드 실행, 자동 완성, 디버깅과 가상환경 선택 지원
  - Jupyter 확장을 통한 Notebook 작성과 실행 지원
  - 필요한 기능만 선택하여 가벼운 개발 환경 구성 가능
- **본 수업은 Anaconda와 VS Code를 연결하여 데이터 분석 실습 진행**
  - Anaconda는 Python, 라이브러리 및 가상환경 관리 담당
  - VS Code는 코드 작성, 실행, 디버깅 및 Notebook 작업 담당
  - 본 수업은 Windows 환경을 기준으로 VS Code 설치 방법 설명

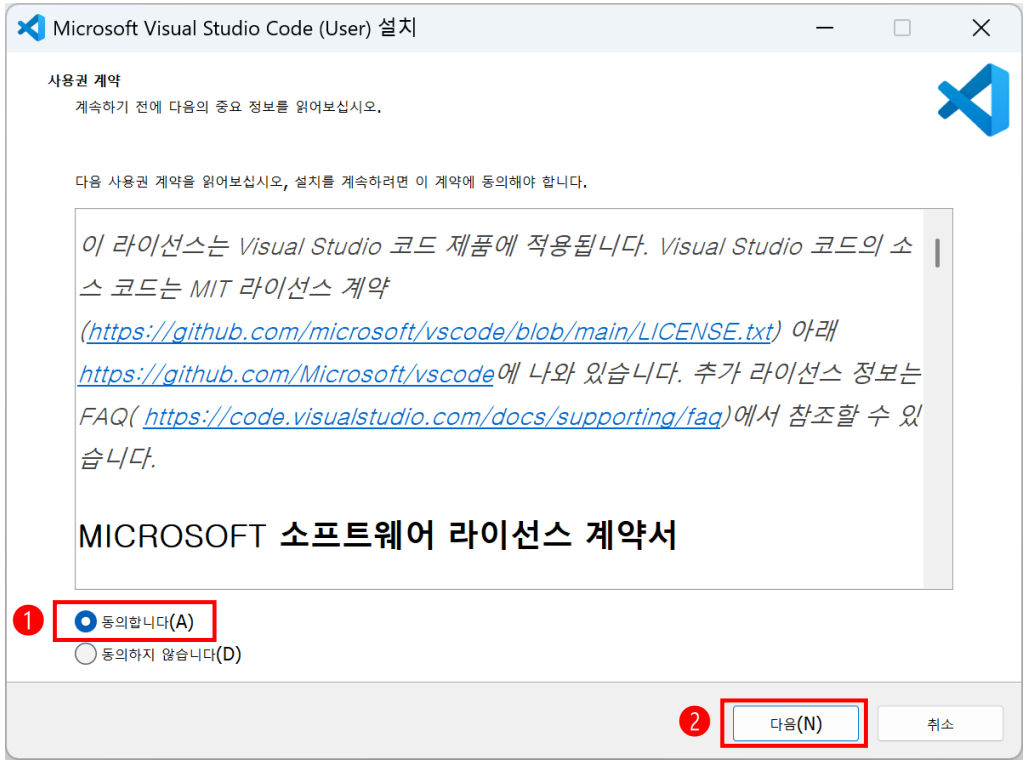
설치 파일 다운로드

- **공식 홈페이지 접속**
  - https://code.visualstudio.com
  - Download for Windows를 선택하여 User Installer 다운로드



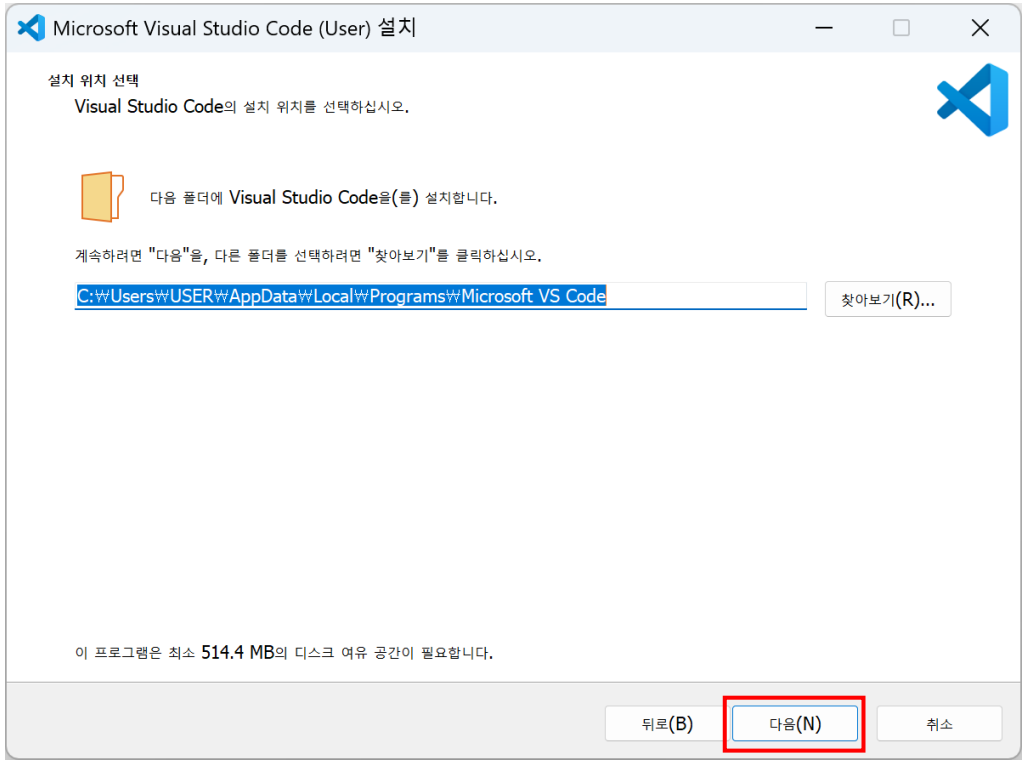
설치 파일 실행

- 다운로드한 VS Code 설치 파일 실행
  - 사용권 계약에서 [동의합니다] 선택
  - [다음]을 선택하여 설치 진행



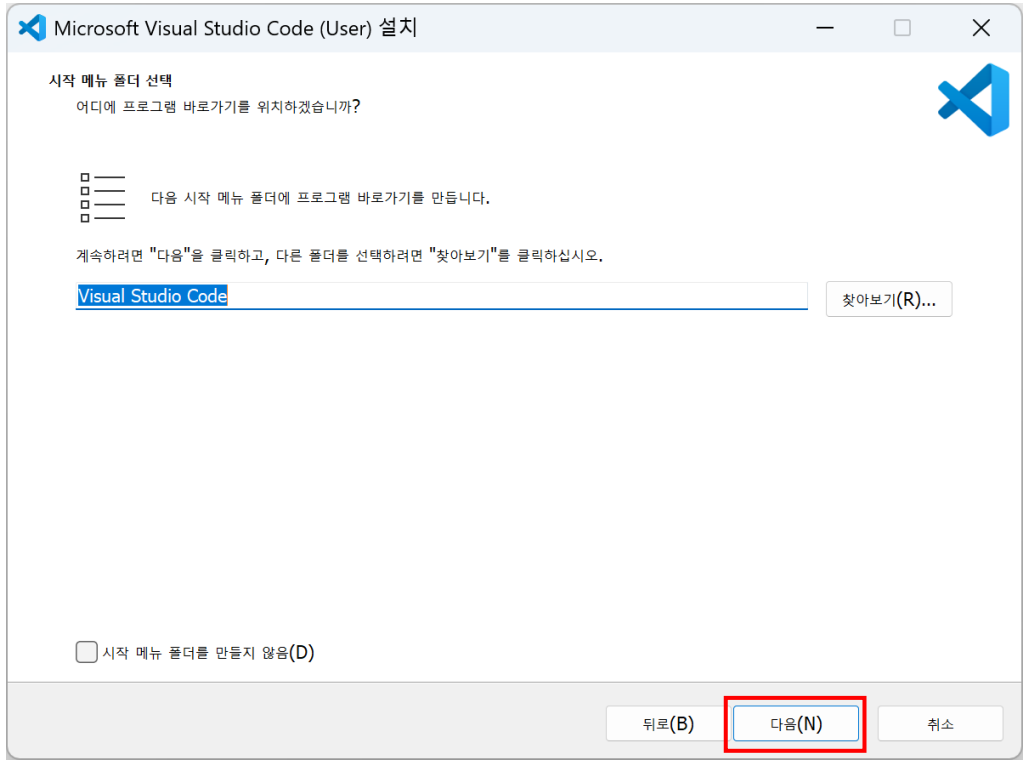
설치 경로 확인

- **기본 설치 경로를 확인한 후 [다음] 선택**
  - 특별한 변경이 필요하지 않으면 기본 경로 사용 권장



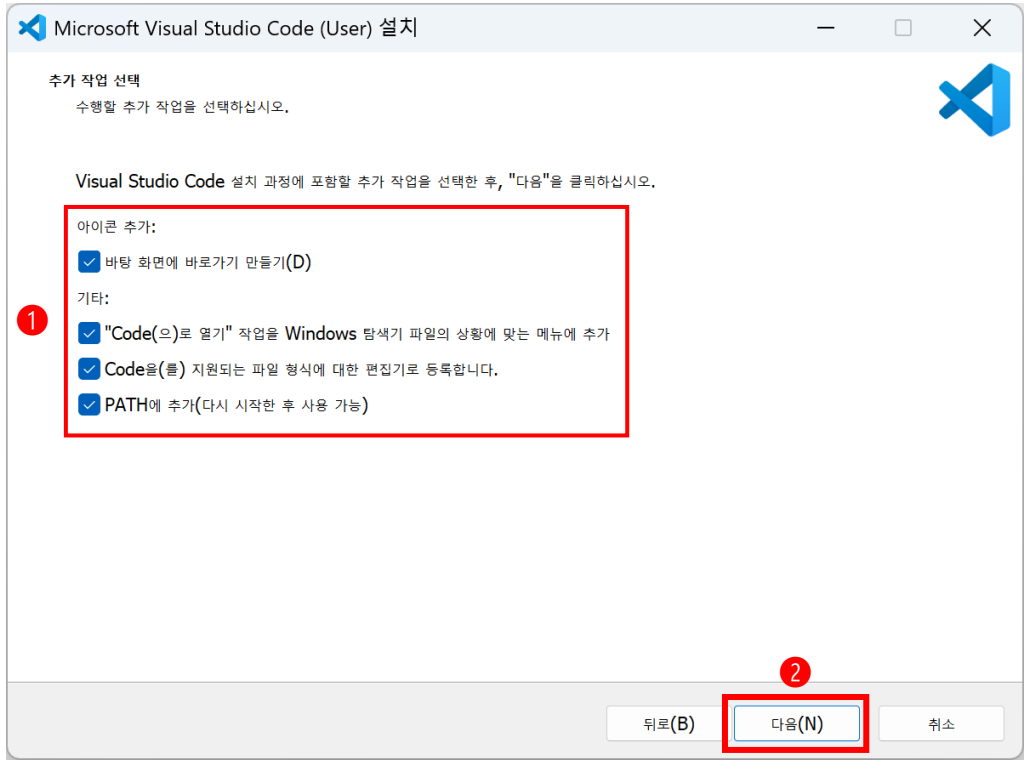
시작 메뉴 폴더 선택

- **기본 시작 메뉴 폴더를 유지한 후 [다음] 선택**
  - 별도의 바로가기 폴더가 필요한 경우에만 이름 변경



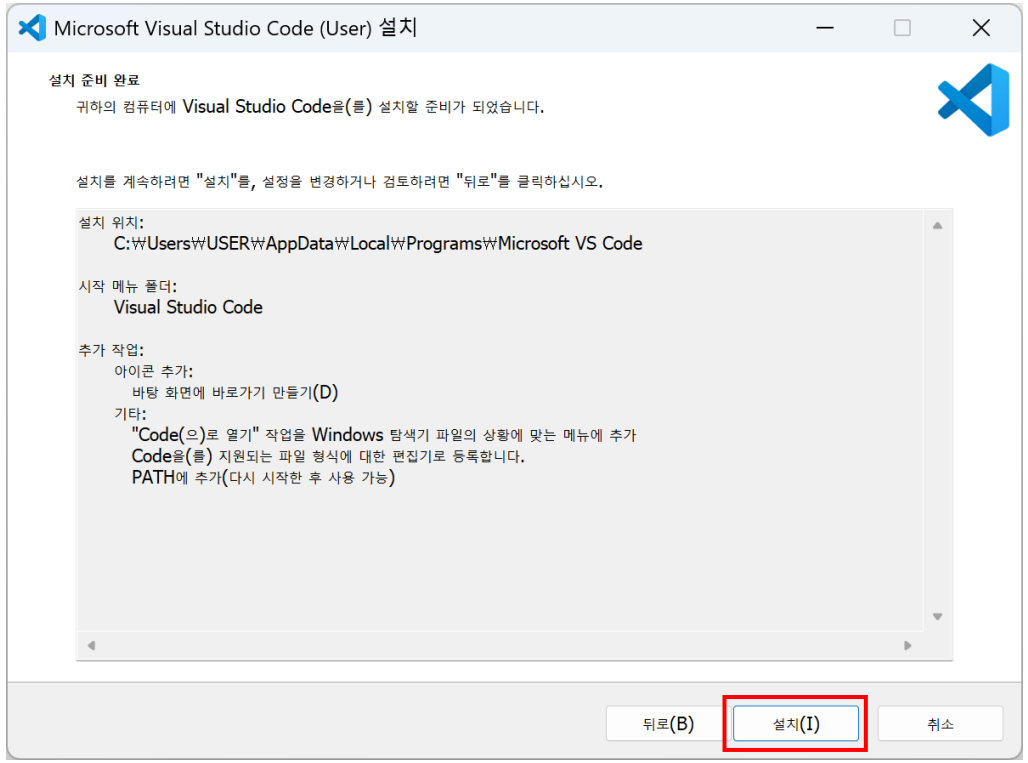
추가 작업 선택

- **실습 편의를 위한 추가 작업을 선택한 후 [다음] 진행**
  - 명령 프롬프트에서 VS Code를 실행할 수 있도록 [PATH에 추가] 선택
  - 탐색기에서 폴더를 바로 열 수 있도록 [Code(으)로 열기] 옵션 선택
  - 바탕화면 바로가기와 파일 연결은 필요에 따라 선택



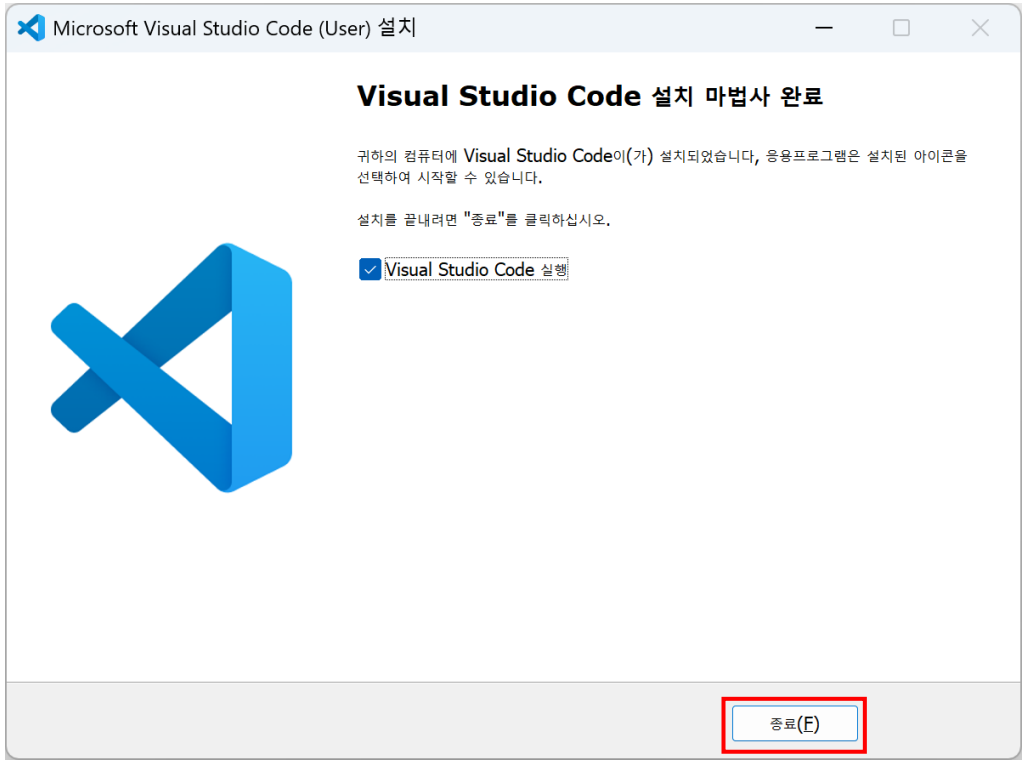
설치 진행

- **설치 설정을 확인한 후 [설치] 선택**
  - 설치가 완료될 때까지 설치 프로그램을 종료하지 않음



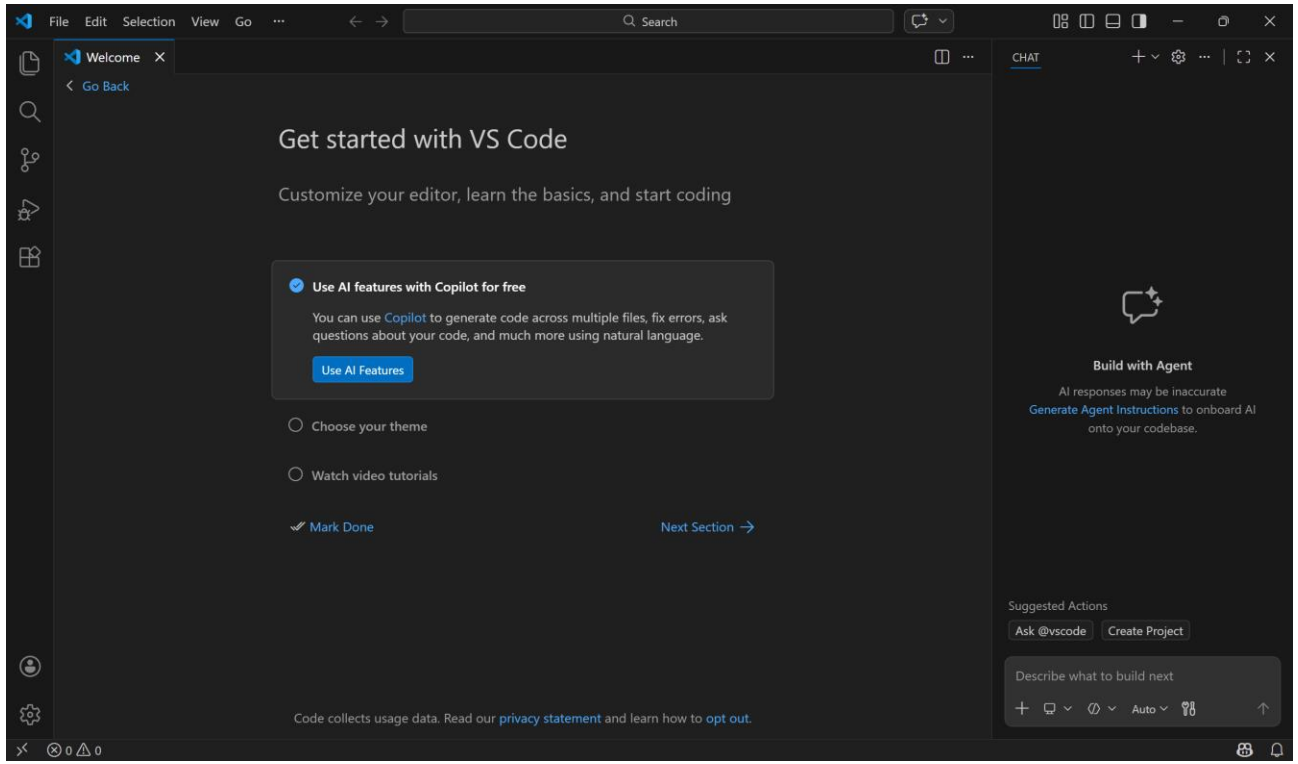
설치 완료

- Visual Studio Code 실행을 선택한 후 [종료] 클릭
  - 설치 종료와 함께 VS Code 자동 실행



VS Code 실행 확인

- **VS Code 시작 화면이 정상적으로 열리는지 확인**
  - AI 코딩 도구의 로그인과 설정은 선택 사항
  - Python 실습 기능은 확장 기능 설치 후 사용 가능

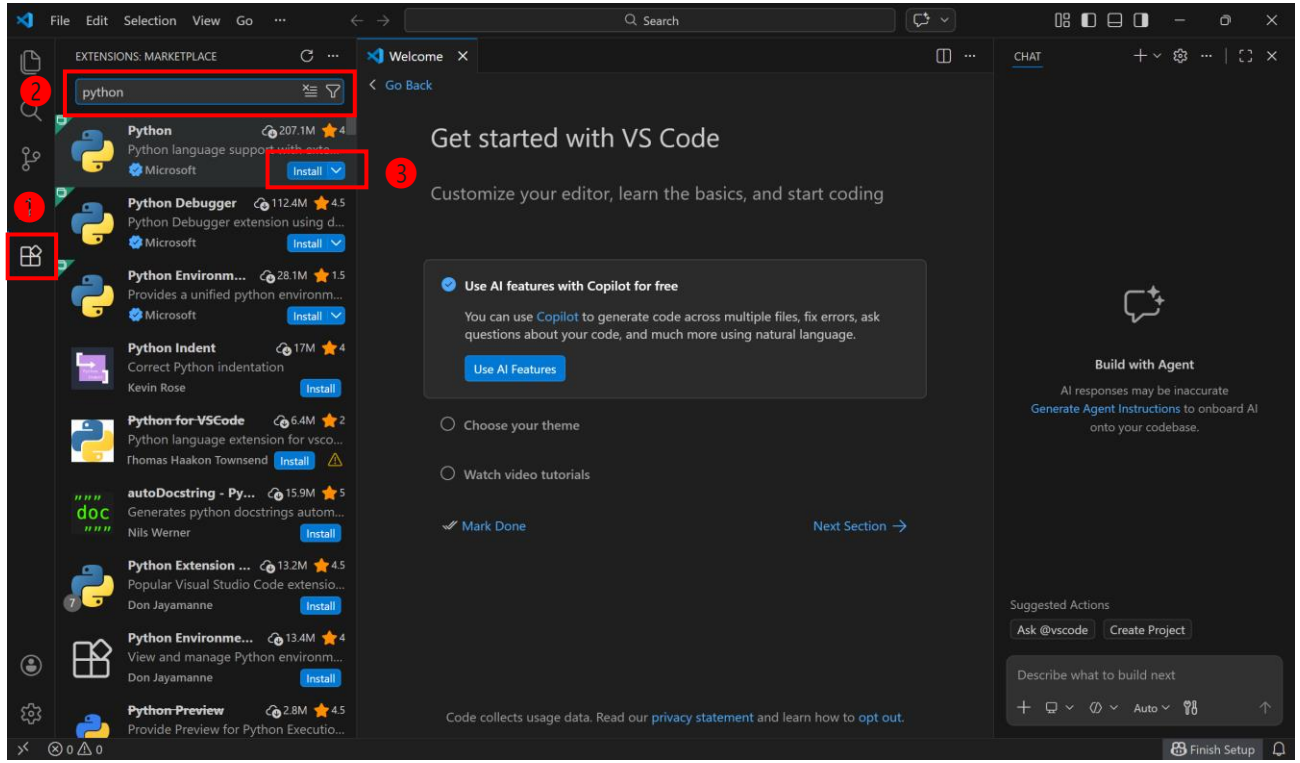


Python 실습용 확장 기능

- **Python 확장은 Python 코드 작성과 실행을 위한 필수 기능 제공**
  - 코드 자동 완성, 실행, 디버깅과 Python 가상환경 선택 지원
  - Python Debugger 등 관련 확장이 함께 설치될 수 있음
- **Jupyter 확장은 Notebook 파일 작성과 실행을 위해 설치 필요**
  - 코드 셀 실행, 실행 결과 확인과 커널 선택 지원
  - 본 수업의 Jupyter Notebook(.ipynb) 실습을 위해 설치 필수
- **Codex, Claude Code 등의 AI 코딩 도구는 필요에 따라 선택적으로 활용**
  - 코드 작성, 코드 설명, 오류 분석과 수정 제안 등 개발 작업 지원
  - 도구에 따라 확장 기능 또는 별도 프로그램 설치와 계정 로그인이 필요
  - 본 수업 실습에 필수적인 도구는 아님

확장 기능 설치

- **Extensions 보기에서 필요한 확장 기능을 검색한 후 [Install] 선택**
  - 왼쪽 활동 표시줄의 Extensions 아이콘을 선택하여 확장 보기 실행
  - Python 확장을 검색한 후 게시자가 Microsoft인지 확인
  - 같은 방법으로 Microsoft에서 제공하는 Jupyter 확장 설치



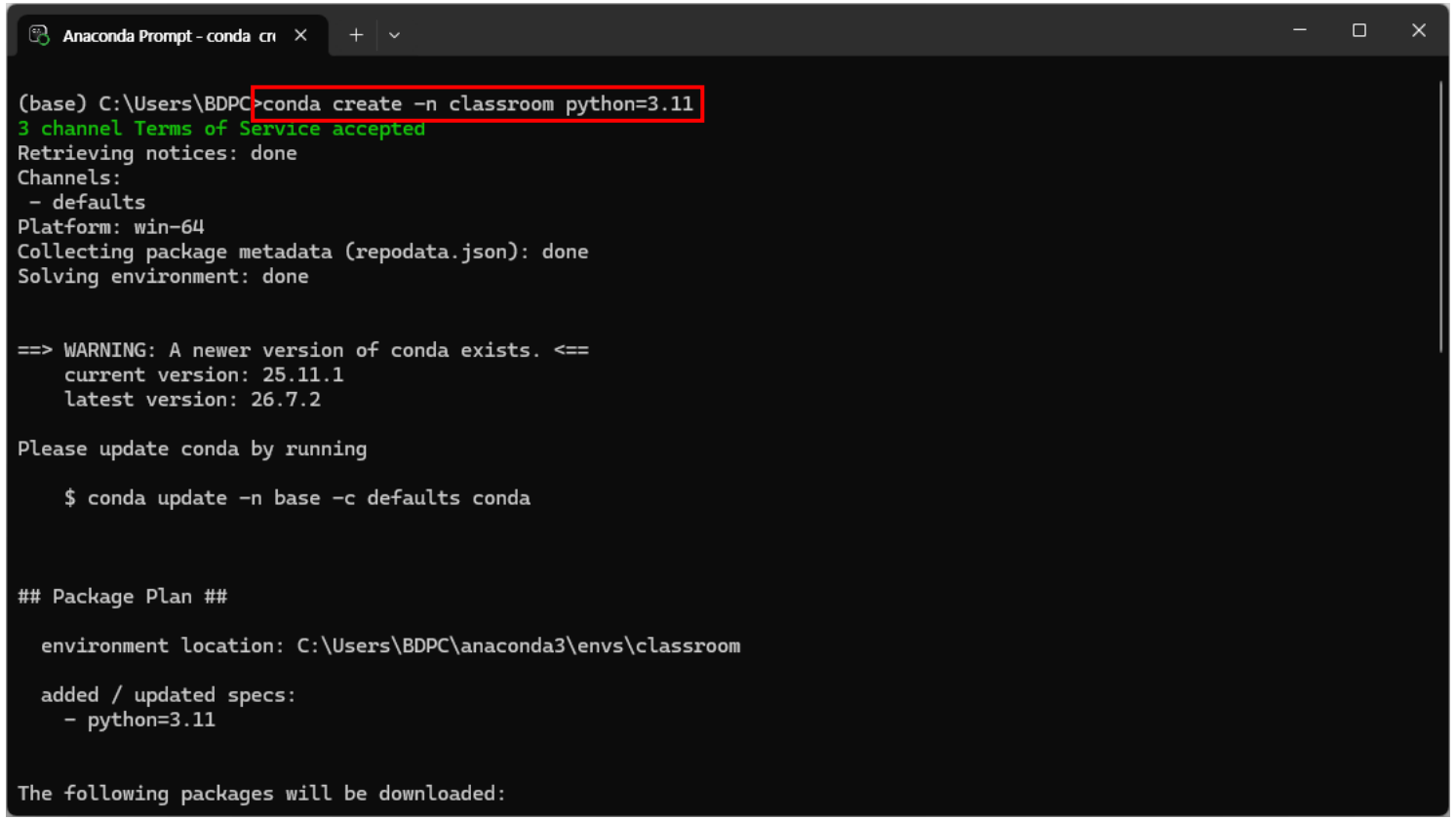
가상환경 설치 방법

가상환경 개요

- **가상환경은 프로젝트별로 독립된 Python 실행 환경을 구성하는 기능**
  - 프로젝트마다 Python과 라이브러리 버전을 구분하여 충돌 방지
  - 다른 프로젝트에 영향을 주지 않고 필요한 패키지 설치 및 관리 가능
- **본 수업에서는 Anaconda에 포함된 conda로 실습 환경 구성**
  - 기본 환경(base)에 직접 설치하지 않고 수업용 환경을 별도로 사용
  - conda를 통해 가상환경과 패키지를 함께 관리

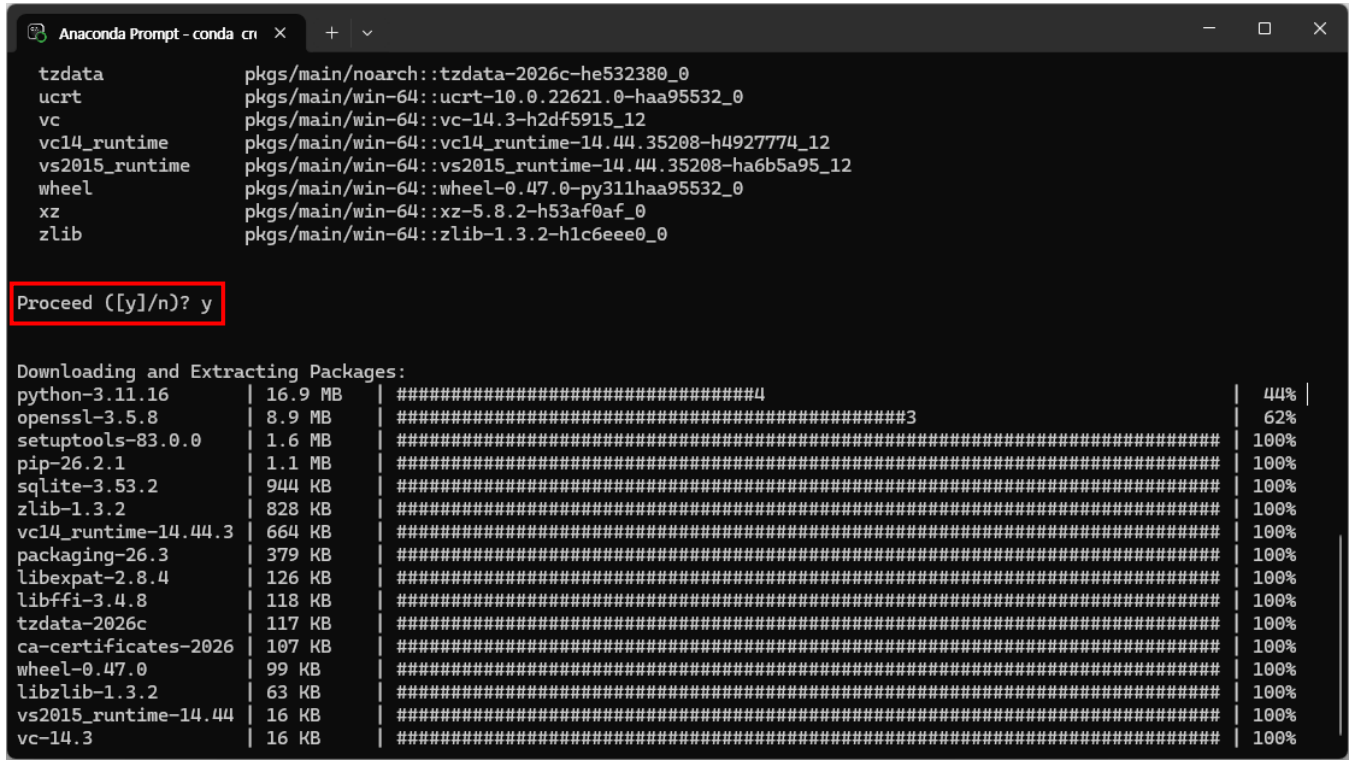
가상환경 생성

- **Anaconda Prompt를 실행한 후 새로운 가상환경 생성**
  - conda create –n 환경이름 python=버전 (예: conda create –n classroom python=3.11)
  - classroom은 가상환경 이름이며 필요한 경우 변경 가능
  - Python 버전은 분석 목적과 패키지 호환성을 고려하여 선택 (※수업에서는 3.12 권장)



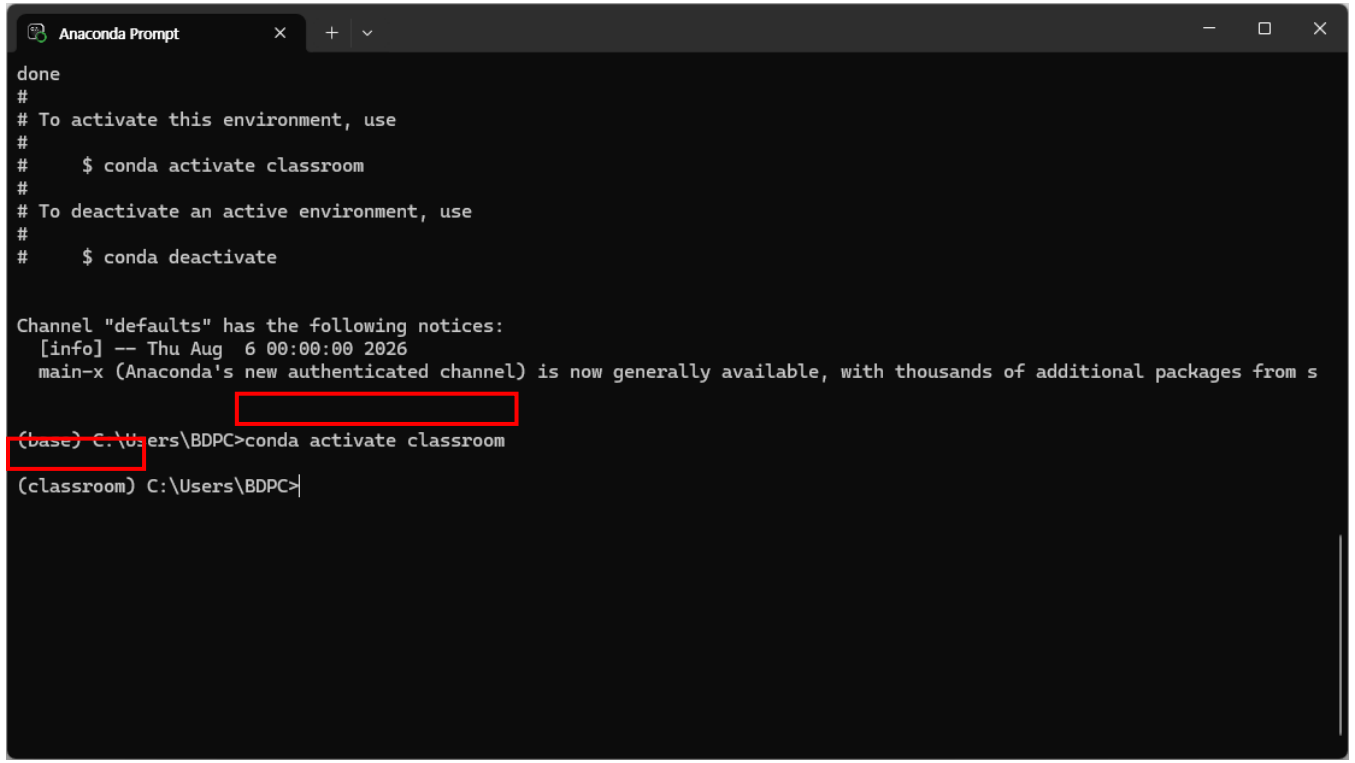
가상환경 생성

- Proceed ([y]/n)?가 나타나면 y 입력 후 Enter 키 누름
  - Python과 기본 패키지 설치가 완료될 때까지 대기



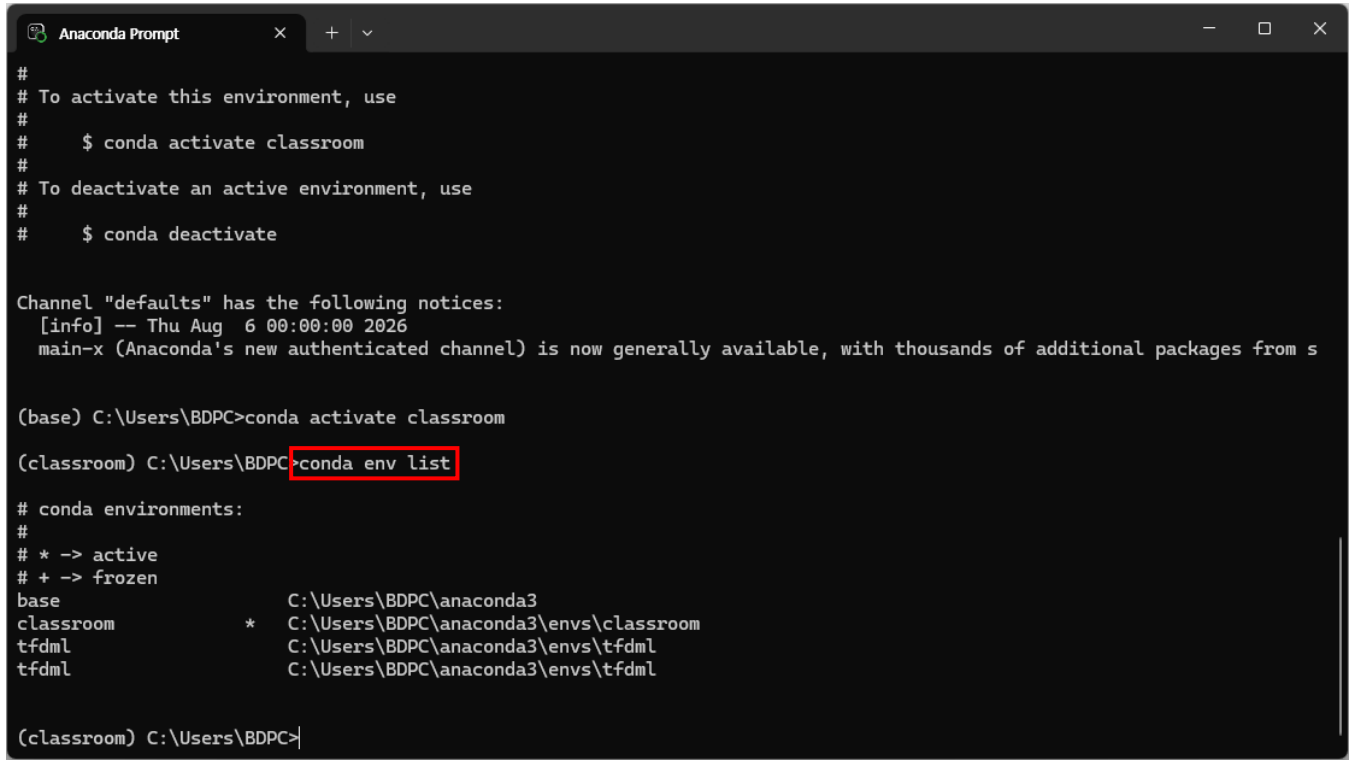
가상환경 활성화

- **생성한 가상환경을 사용하기 위해 활성화 명령어 실행**
  - conda activate 환경이름 (예: conda activate classroom)
  - 활성화되면 명령 프롬프트 앞부분에 (환경이름) 표시
  - 현재 가상환경 종료: conda deactivate



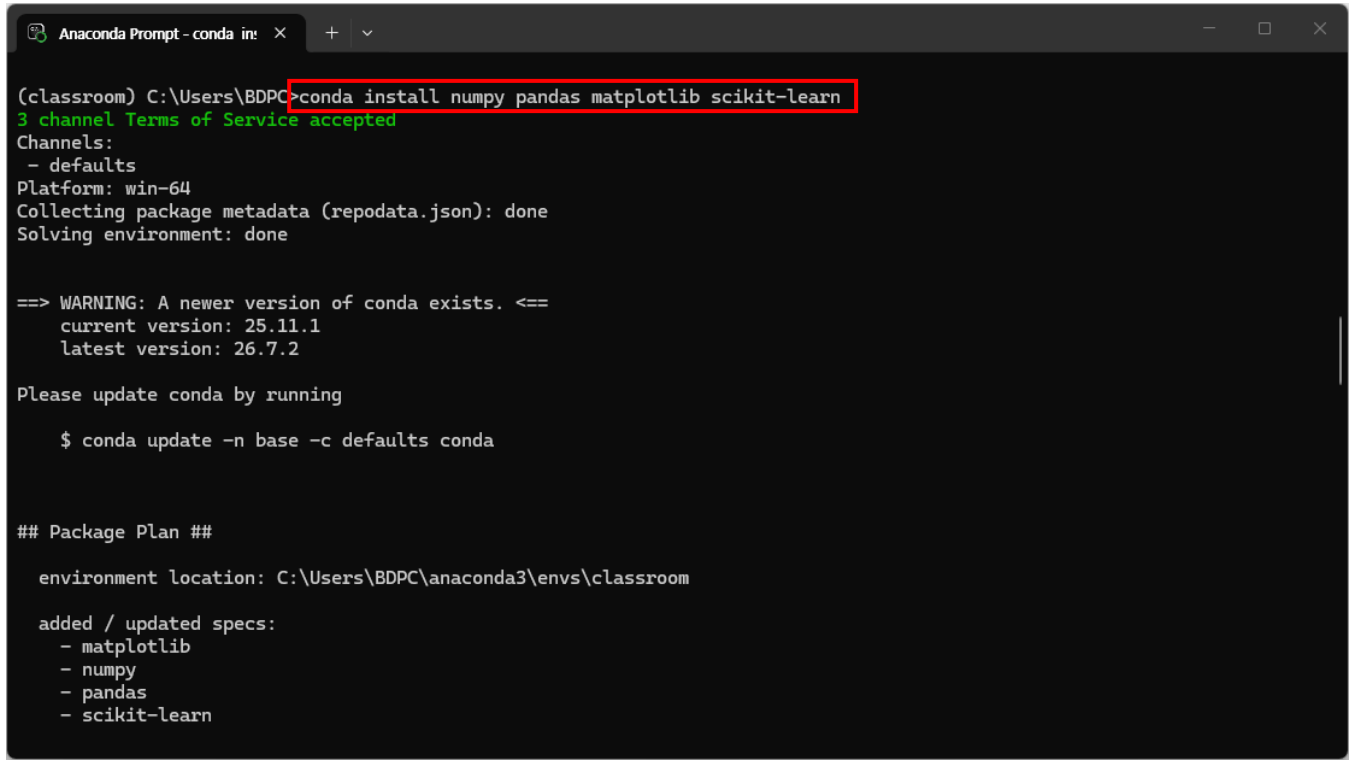
가상환경 확인

- **생성된 가상환경의 목록과 활성화 상태 확인**
  - conda env list
  - 현재 활성화된 가상환경은 이름 옆에 * 표시
  - 사용하지 않는 가상환경 삭제: conda remove –n 환경이름 ––all



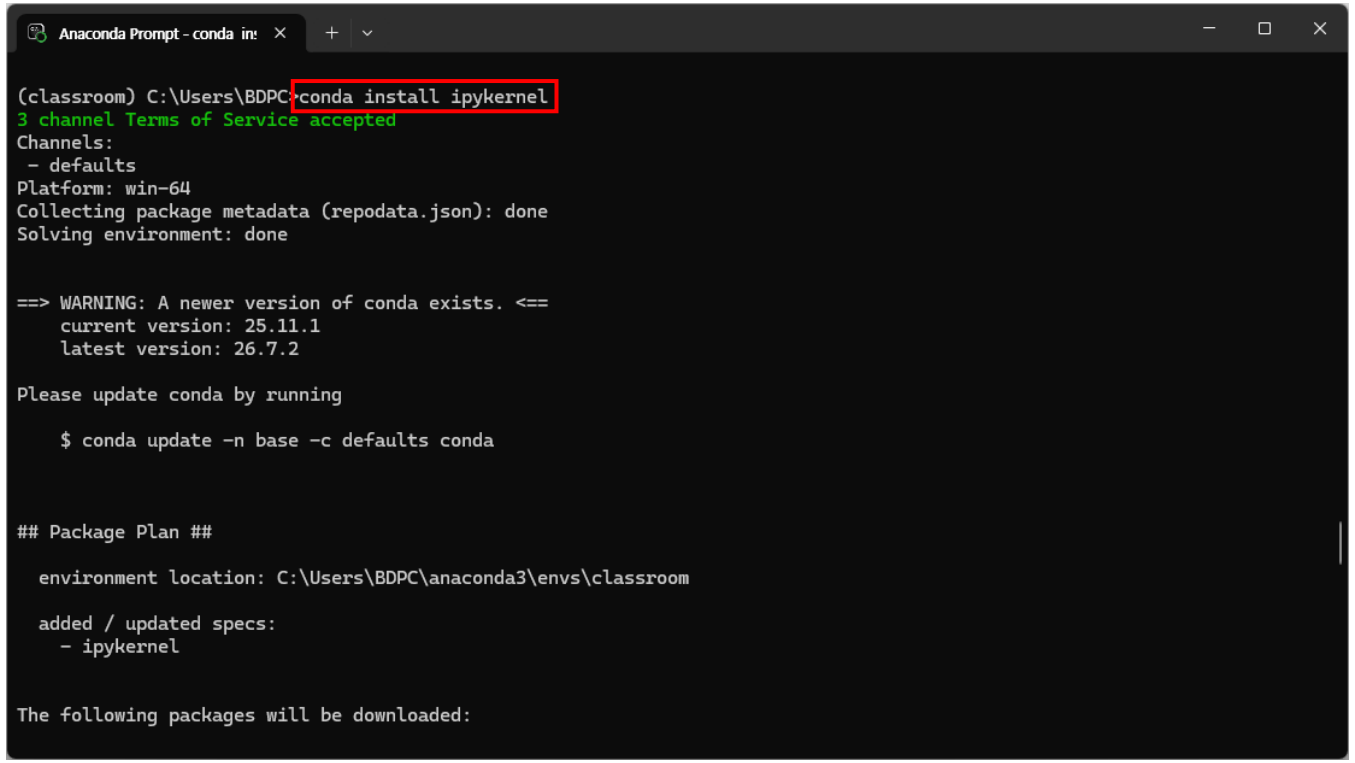
필요한 라이브러리 설치

- **가상환경을 활성화한 상태에서 conda로 필요한 라이브러리 설치**
  - conda install numpy pandas matplotlib scikit-learn (데이터 분석에 필요한 기본 패키지 설치)
  - conda에서 제공하지 않는 패키지는 필요한 경우 pip로 설치 가능
  - ※ 설치 확인 메시지가 나타나는 경우에만 y 입력 후 Enter



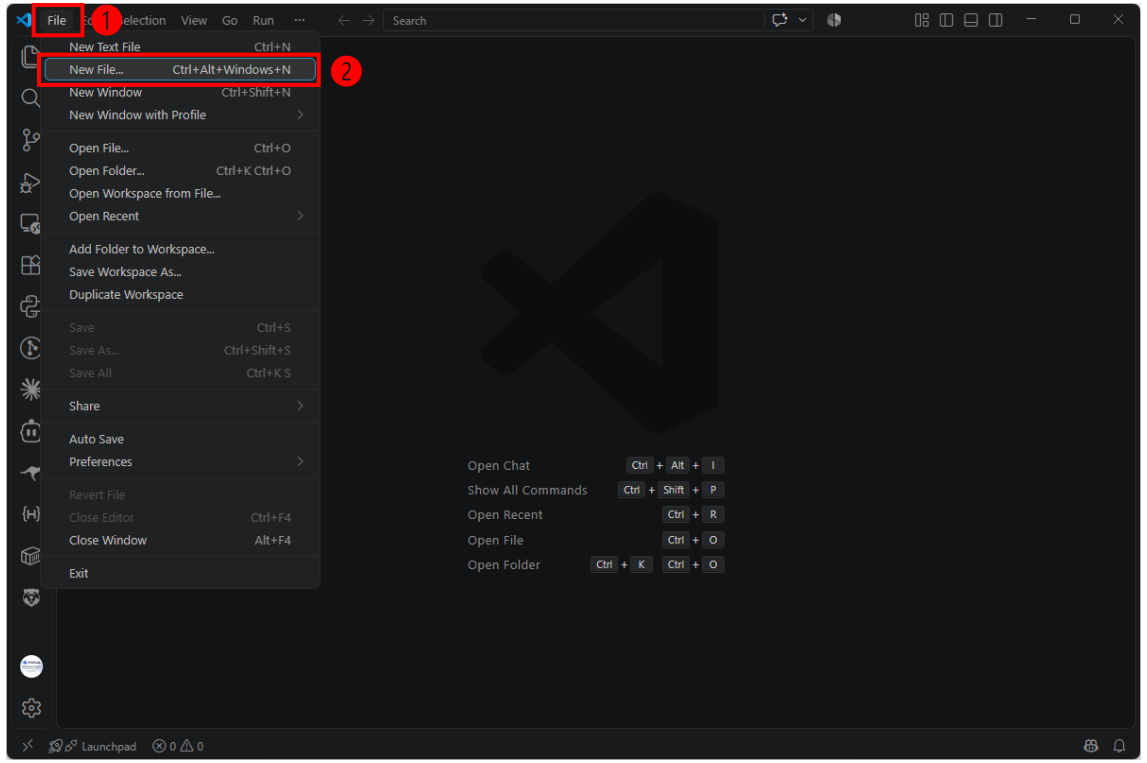
ipykernel 설치

- 가상환경을 Jupyter Notebook에서 사용하기 위해 ipykernel 설치
  - conda install ipykernel
  - 필요한 경우 pip install ipykernel 명령어로도 설치 가능
  - ※ 설치 확인 메시지가 나타나는 경우에만 y 입력 후 Enter



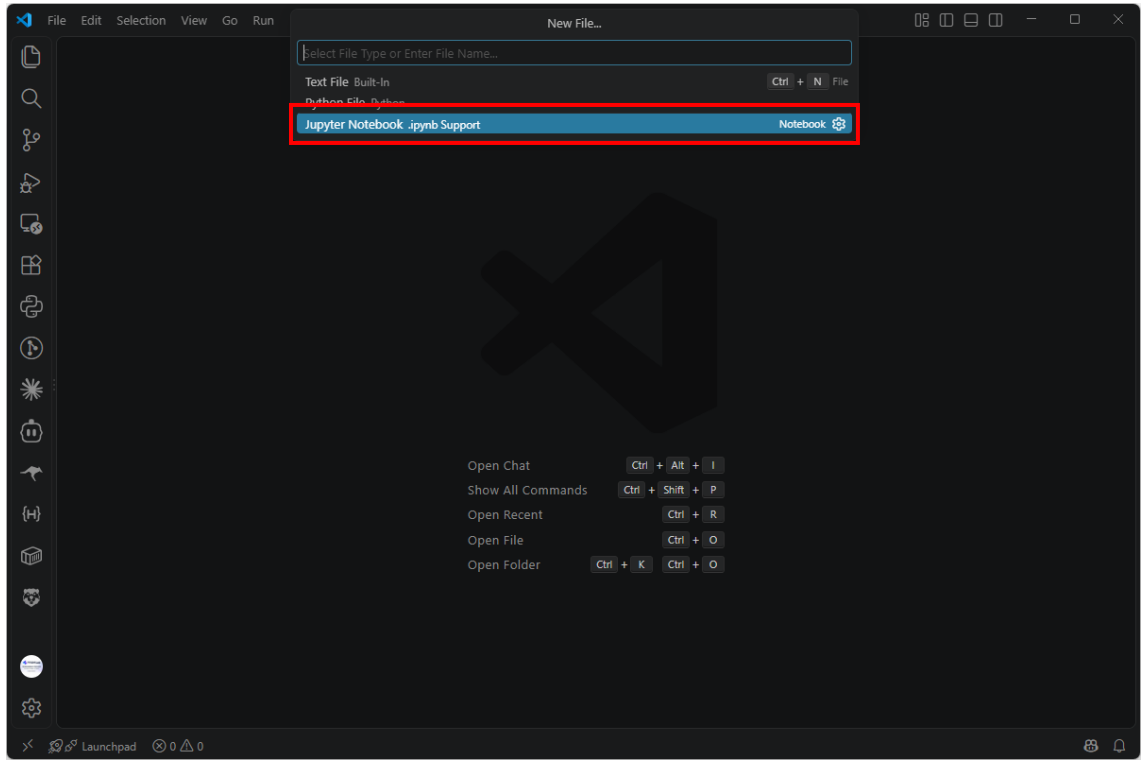
가상환경 선택

- VS Code의 [File] 메뉴에서 [New File] 선택



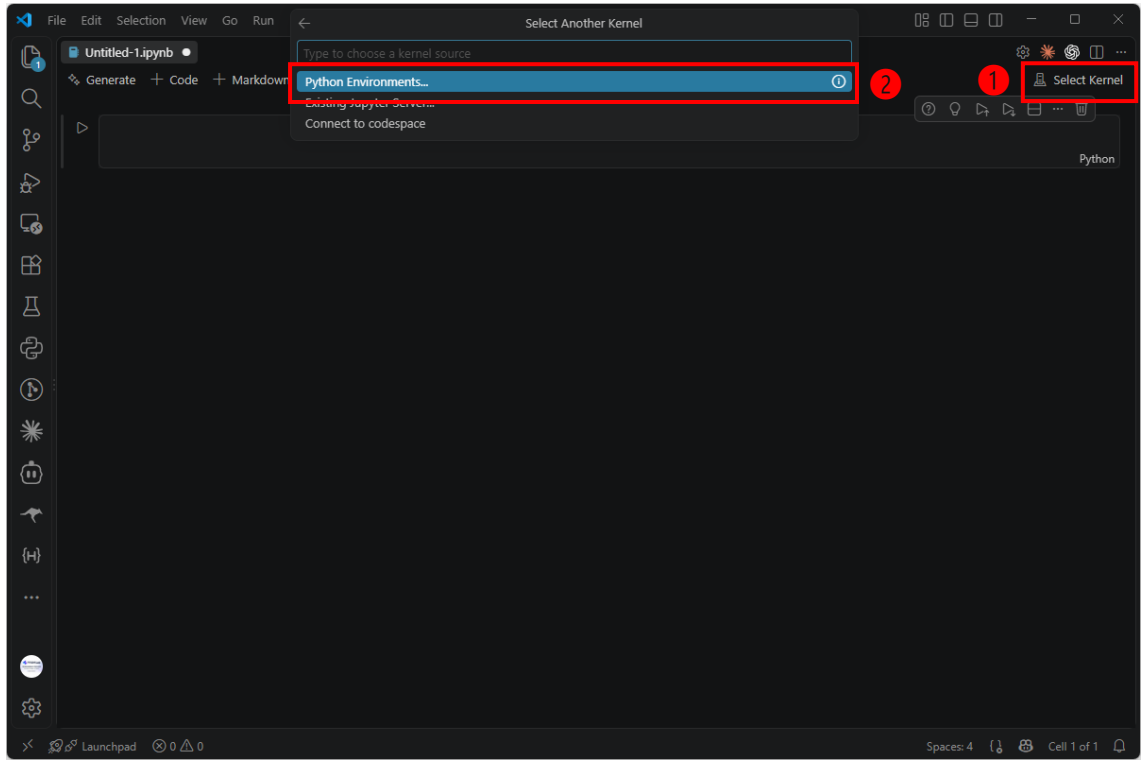
가상환경 선택

- **파일 유형에서 [Jupyter Notebook] 선택**
  - 새로운 Notebook 파일이 열리면 가상환경 선택 단계로 이동



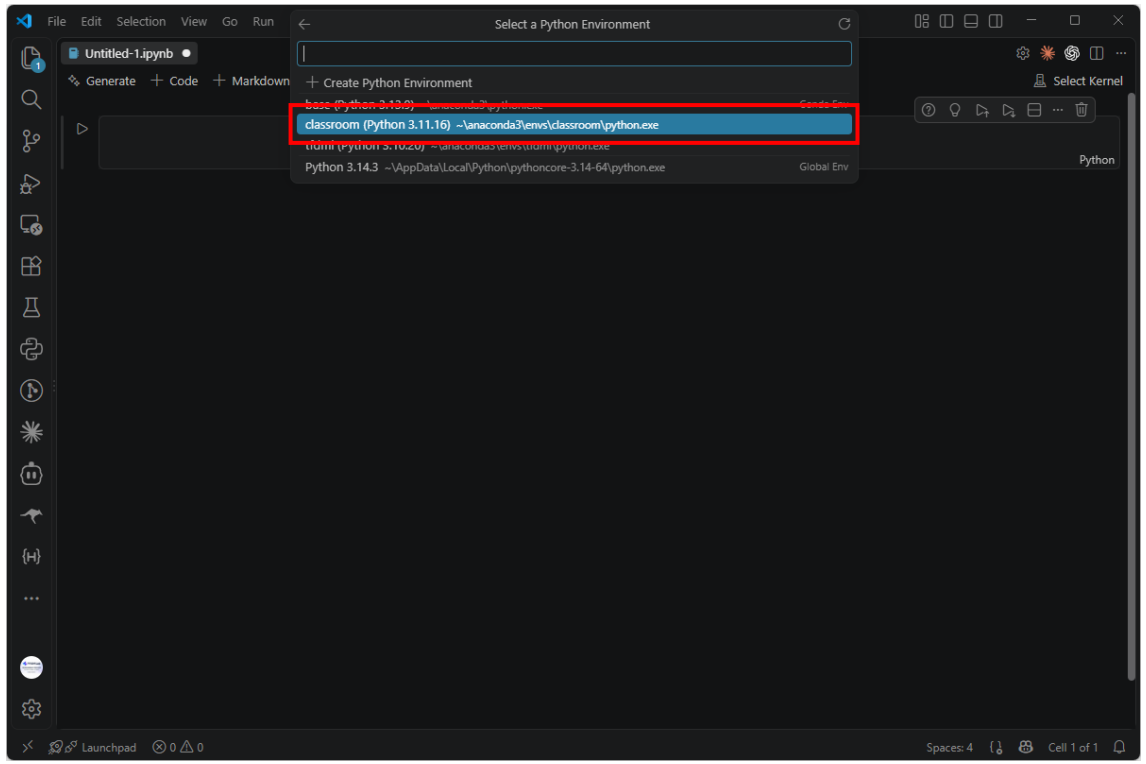
가상환경 선택

- Notebook 오른쪽 위의 [Select Kernel] 선택
  - 커널 선택 메뉴에서 [Python Environments] 선택



가상환경 선택

- **등록된 가상환경 목록에서 사용할 환경 선택**
  - 확장 기능 설치 및 커널 검색 안내가 나타나면 완료될 때까지 대기
  - 환경이 표시되지 않으면 Python 및 Jupyter 확장 기능을 확인한 후 VS Code 재실행



HSU HANSUNG UNIVERSITY

수고하셨습니다